

MODULE 14 · DOMAIN 4 - PROMPT ENGINEERING & STRUCTURED OUTPUT

Cost, Latency and the Batch API

One question decides most of it: is anybody waiting for this answer?

DOMAIN 4 of 5 EXAM WEIGHT 20% MODULE 14 of 20

A user staring at a screen needs a synchronous call. A user who submitted a job and walked away does not.

What the Batch API gives you

PROPERTY	DETAIL
Cost	50% off standard rates, on input and output tokens
No threshold	The discount is unconditional. A batch of ten gets it.
Turnaround	Up to 24 hours. Most complete in minutes to hours.
Scale	Up to 100,000 requests, or 256 MB, per batch
Keying	Each request carries a custom_id; results are keyed to it
Delivery	Results as JSONL, retained for 29 days

ON THE EXAM

Half price for asynchronous processing. The only thing you give up is knowing when it will finish.

Operational details that catch people

- ◆ **There are no webhooks.** You poll for status, so build for that.
- ◆ **Results expire after 29 days.** Copy them to your own storage as soon as the batch completes.
- ◆ **custom_id is how you match a result to a record.** Make it your internal identifier, not a row number.
- ◆ **Cancellation is best-effort.** Requests already in flight may still complete, and you are billed for them.

ON THE EXAM

Batch is an asynchronous system with an asynchronous system's obligations. Poll, persist, and match by ID.

Failures inside a batch

- ◆ **Each result is individually succeeded or errored.** One bad request does not fail the batch.
- ◆ **Handle errors at the row level**, and resubmit only the failures.
- ◆ **A request that expires past the 24-hour window is not billed**, and produces no result either.
- ◆ **Errors arrive hours later, in bulk.** You lose the immediate feedback a synchronous call gives you.

ON THE EXAM

Resubmit the failures, not the batch. Reprocessing ten thousand successes to fix forty errors is pure waste.

Batch or synchronous?

BATCH

Nobody is waiting

- Nightly jobs, backfills, bulk classification
- Evaluations and offline analysis
- Content moderation queues, data labelling
- Cost matters and latency does not

SYNCHRONOUS

Someone is waiting

- Anything in a live conversation
- A customer expecting a reply now
- An agent loop that needs the next result
- Latency is part of the requirement

THE TIEBREAKER

An agentic loop cannot use batch. Each step depends on the previous result, so a 24-hour turnaround per step is not a system.

Routing, rather than choosing

- ◆ **Most workloads are not uniformly urgent.** A minority needs a reply now; the rest can wait.
- ◆ **Route the urgent slice synchronously and the remainder to batch.** One system, two paths.
- ◆ **The classifier deciding which path is cheap** and can often be a rule rather than a model.
- ◆ **The saving is proportional to how much of the volume can wait**, which is usually most of it.

ON THE EXAM

"Batch or synchronous" is often a false choice. The strongest answer routes by urgency instead of picking one.

Parallelism buys **wall-clock time**. Batch buys **money**. They are not substitutes.

Prompt caching, and how it stacks

- ◆ **A long shared prefix** — a system prompt, a document, a set of examples — can be cached and reused.
- ◆ **Cache reads cost far less than fresh input tokens.** You still pay to write the cache.
- ◆ **It works inside a batch too**, and the batch discount applies to cache pricing as well.
- ◆ **It only helps when the prefix is genuinely shared and stable.** A prefix that changes every call caches nothing.

ON THE EXAM

Caching and batching are independent levers and they compound. Reach for both when the workload allows.

Truncation, one more time

- ◆ **stop_reason of max_tokens means the output hit the ceiling you set.** The JSON is cut mid-structure.
- ◆ **Resending identically fails identically.** It is deterministic, not transient.
- ◆ **Raise the budget when the output was just over.**
- ◆ **Split the work when no reasonable budget would fit it,** extracting in sections and combining afterwards.
- ◆ **Do not confuse it with the context window being exceeded.** Opposite fixes, as module 1 covered.

ON THE EXAM

Raising the ceiling handles the near misses. Splitting handles the documents that would overflow any ceiling.

The levers that actually move cost

- ◆ **Batch the work that can wait.** Half price, for a scheduling change.
- ◆ **Cache the shared prefix.** Free money when the prefix is stable.
- ◆ **Send less.** Tool definitions, examples and history are all input tokens on every call.
- ◆ **Match the model to the task.** A classification step does not need your most capable model.
- ◆ **Scope the task properly.** Cost overruns usually come from one task iterating far more than expected.

ON THE EXAM

Volume is rarely the problem. A badly scoped task that loops thirty times is the problem.

Reading the stem: cost and scale questions

WHAT THE STEM SAYS	WHAT IT WANTS
"reviewed the next morning", "nightly", "backfill"	Batch. Nobody is waiting.
"the customer is waiting for a reply"	Synchronous. Latency is the requirement.
"most requests can wait, some cannot"	Route by urgency. Both paths.
"reduce cost" with parallel execution offered	Parallelism does not reduce cost. Look elsewhere.
"truncated mid-JSON", "retries fail identically"	max_tokens. Raise it, or split the work.
"forty of ten thousand failed"	Resubmit the failures, not the batch.

ON THE EXAM

Find the sentence about who is waiting. It decides more of these than any other detail.

Module 14 in five lines

- ◆ Batch is 50% off, unconditionally, for up to 24 hours of patience.
- ◆ Ask who is waiting. Nobody means batch; a person on a screen means synchronous.
- ◆ An agentic loop cannot use batch. Each step needs the previous result.
- ◆ Parallelism buys time. Batch buys money. Caching stacks with batch.
- ◆ Resubmit the failures, not the batch. Errors are per-row.

ON THE EXAM

That closes Domain 4. Five scenarios, then Domain 5 and context management.

A compliance team processes around 40,000 documents each month for policy classification. Results are reviewed by analysts during their weekly Monday meeting.

The pipeline currently sends every document as a synchronous request as it arrives. The monthly bill has become a concern.

No individual document result is used before the Monday review.

What is the most effective change?

- ☐ A Send the documents through the Message Batches API
- ☐ B Process the documents in parallel to reduce total processing time
- ☐ C Switch to a smaller model for all classifications
- ☐ D Reduce max_tokens so responses are shorter

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Batch API
Nothing is used before Monday, so latency is worth nothing here. Half price for a scheduling change.
- B

 Parallel processing
Buys wall-clock time they do not need, and costs exactly the same.
- C

 Smaller model
Might work, and it trades accuracy for cost with no evidence the smaller model is adequate.
- D

 Shorter responses
Output tokens on a classification are a small share of the bill. Input is where the volume is.

THE SIGNAL

"No result is used before the Monday review" is the planted detail. It says explicitly that latency has no value.

A customer support product answers questions in a live chat widget. Median response time is 2.4 seconds and customers abandon the chat beyond about 8 seconds.

The finance team has noticed the Batch API's 50% discount and asks why it is not being used. Volume is around 200,000 conversations per month.

How should you respond?

- ☐ A Batch cannot be used for live chat. Consider prompt caching on the shared system prompt, and a smaller model for simple queries
- ☐ B Use batch for the conversations, since most complete within minutes anyway
- ☐ C Use batch during off-peak hours when customers are less likely to be waiting
- ☐ D Split each conversation so the first message is synchronous and the rest are batched

◆ PAUSE AND DECIDE

How should you respond?

- A

 Not batch; cache and model choice
Says no to the wrong lever and offers two that fit a latency-bound workload.
- B

 Most finish in minutes
Typical is not guaranteed. The SLA is 24 hours, and an 8-second abandonment threshold cannot absorb minutes.
- C

 Batch off-peak
Off-peak customers are still customers, and they are still waiting.
- D

 Batch later messages
The second message in a conversation is more urgent than the first, not less.

THE SIGNAL

"Most batches finish in minutes" is true and irrelevant. You design against the SLA, not the median.

A document extraction batch of 12,000 records completes. The results file shows 11,840 succeeded and 160 errored, mostly on unusually long documents.

An engineer proposes resubmitting the entire batch after raising `max_tokens`, to keep the pipeline simple.

Each record costs roughly the same to process.

What should they do instead?

- ☐ A Resubmit the whole batch as proposed, since simplicity is worth the cost
- ☐ B Resubmit only the 160 failed records, matched by `custom_id`, with a higher `max_tokens`
- ☐ C Switch the whole pipeline to synchronous requests so errors surface immediately
- ☐ D Raise `max_tokens` for future batches and accept the loss of these 160 records

◆ PAUSE AND DECIDE

What should they do instead?

A Resubmit everything

Reprocesses 11,840 successful records to fix 160. Simplicity bought at seventy-four times the necessary cost.

C Go synchronous

Doubles the price of the whole workload to improve error visibility on 1.3% of it.

B Resubmit the failures only

Errors are per-row and custom_id matches results to records. That is exactly what it is for.

D Write off the failures

160 documents silently unprocessed. In extraction, missing records are usually not acceptable.

THE SIGNAL

Per-row error reporting exists so you can resubmit selectively. Not using it is the whole mistake here.

An internal tool answers questions about a large policy handbook. Every request sends the full handbook, about 90,000 tokens, followed by a short question.

The handbook changes roughly twice a year. Around 3,000 questions are asked per day, and users expect an answer within a few seconds.

Input token costs dominate the bill.

What is the most effective change?

- ☐ A Move the workload to the Batch API for the 50% discount
- ☐ B Cache the handbook prefix so repeated questions read from the cache instead of resending it
- ☐ C Summarise the handbook and send the summary instead
- ☐ D Reduce max_tokens on the response

◆ PAUSE AND DECIDE

What is the most effective change?

A Batch it

Users expect an answer in seconds. The one thing batch requires is the one thing this workload cannot give.

C Send a summary

Cheaper and lossy. Policy answers depend on exact wording, so this trades correctness for cost.

B Cache the prefix

A large, stable, shared prefix in front of a short question is the textbook caching case.

D Shorter responses

The stem says input costs dominate. Trimming output addresses the smaller half.

THE SIGNAL

Large prefix, changes twice a year, three thousand calls a day, short question. Every detail in that sentence points at caching.

A moderation system reviews user-submitted content. About 8% of submissions are flagged by an upstream rule as potentially urgent and must be actioned within minutes. The remaining 92% are routine and are reviewed by a human team the following day.

Everything currently goes through synchronous requests. Volume is 500,000 items per month. The team has been asked to reduce cost without slowing the urgent path.

What should they do?

- ☐ A Move everything to the Batch API and accept the delay on urgent items
- ☐ B Keep the urgent 8% synchronous and send the remaining 92% through the Batch API
- ☐ C Keep everything synchronous and use a smaller model throughout
- ☐ D Batch the urgent items overnight and process routine items synchronously during the day

◆ PAUSE AND DECIDE

What should they do?

- A

 Batch everything
Violates the stated constraint. Urgent items need minutes, and batch offers up to 24 hours.
- B

 Route by urgency
92% of the volume gets half price and the urgent path is untouched. The constraint is respected exactly.
- C

 Smaller model everywhere
Applies a quality trade to urgent moderation, which is where accuracy matters most.
- D

 Batch the urgent ones
Exactly backwards, and it is worth noticing how plausible a reversed answer can look at speed.

THE SIGNAL

The upstream rule already sorts the traffic. The routing decision is free, which is what makes B so strong here.